PAD: Separating the Stateless Brain from the Stateful Hub for Heterogeneous Long-Horizon Agent Fleets

Yanming Liu [1] **Xinyue Peng** [2]

Abstract

Long-horizon agent systems fail less from weak reasoning than from a runtime that loses track of what was asked, what has been done, and what counts as evidence that it was done. We present **PAD**, a hub that owns all durable task state, routes work by declared *capability* rather than by machine identity, and refuses to mark a task done unless the executor submits machine-checkable evidence for each acceptance criterion. A two-month deployment of six heterogeneous nodes—938 work tasks over 835 machine-hours, the longest running 42 hours—produced per-criterion evidence in 93% of completed tasks and a structured report in 42% of failed ones. It also showed that half of all failures are executor-environment faults rather than reasoning failures, which a hub can see and a single-loop harness cannot.

1. Introduction

A language model that can call tools is not yet a system that can be trusted with a week of work. The gap is not reasoning quality but *state management*. When a task spans several hours, several machines, and several failure modes, the questions that decide success are mundane: which subtask is currently running, on which host, under which credentials, and what evidence exists that step three actually happened rather than merely being reported as happened.

The agent-harness literature has largely answered this by enlarging the container. Harnesses keep a growing transcript, add planning and audit roles inside the same loop, or let an agent spawn recursive copies of itself with the same filesystem (Ma et al., 2026; Lumer et al., 2026; Zheng et al., 2026). This works well when the execution environment is one machine and the task fits in one context. It degrades predictably when the work is distributed: the transcript becomes a log of one host's view, and a self-assessment made inside that loop propagates into later decisions (Ma et al., 2026). A recent survey of persistent-state agents makes the same diagnosis from the other end—the literature concentrates on *accumulating* state and much less on governing, recovering, and relinquishing it (Ding et al., 2026).

This report describes a system built on the opposite premise. We separate two things that mainstream harnesses fuse:

- a **stateless brain**—a general chat model that can only ask "what is the situation" and "do this, and here is what would count as done";
- a **stateful hub**—a single process with a single database that owns every durable fact: who is alive, what is queued, who claimed what, what came back, and what evidence accompanied it.

The brain never holds a host name, never issues a shell command, and never sees an executor's log. The hub never reasons, never plans, and never retries with judgment. Everything that varies—models, CLIs, operating systems, credentials—lives on the executor side behind a declarative capability table.

Contributions.

1. **An architecture** (§3) that makes the model's interface to a fleet nine tools wide, and makes topology changes invisible to it.
2. **A dispatch contract** (§3.4) in which tasks are goals with acceptance criteria, and completion is gated on evidence per criterion rather than on an exit code.
3. **A deployment study** (§5) over 938 real work tasks and 835 machine-hours on six heterogeneous hosts, quantifying evidence coverage, failure-report quality, cancellation propagation, and where the design does *not* pay off.
4. **A failure taxonomy** (§6) from 131 failed tasks, with the practical lesson that most failures are executor-environment failures, not reasoning failures—which is

[1]Zhejiang University, Hangzhou, China [2]Southeast University, Nanjing, China. Correspondence to: Yanming Liu <oceann24@zju.edu.cn>, Xinyue Peng <xinyuepeng@seu.edu.cn>.

System report in ICML 2025 layout; not a conference submission.

precisely what a hub can see and a single-loop harness cannot.

What this is not. PAD is not a new agent algorithm and does not improve any model. It is an argument about where state should live, with an implementation and evidence. We position it against harness work in §2 and are explicit in §7 about the three costs it introduces.

2. Related Work

Harness engineering. The framing that agent capability is increasingly *externalized* into memory, skills, protocols, and runtime scaffolding rather than into weights is now well established (Zhou et al., 2026). DeepSeek Harness takes this to its logical end for a single execution environment: everything, including the agent loop and the model adapter, is a replaceable plugin composed at boot (DeepSeek-AI, 2026; Cordis Authors, 2026). That design optimizes *extensibility within one host*. PAD optimizes *routing across many hosts*; the two are complementary, and in our deployment the per-node executor is exactly such a harness CLI, invoked headlessly.

Long-horizon state. LongHorizon-Harness reformulates long-horizon execution as task-state management, keeping state outside execution and updating it only with facts verified from the environment, via a manage–execute–audit loop (Ma et al., 2026). OneDayAgent argues that goal drift, state loss, and context overflow must be handled jointly by one harness (Zheng et al., 2026). We share the diagnosis and differ in placement: they keep the state inside the harness that also executes; we put it in a hub that does not execute at all, so that a node crash cannot take the ledger with it. Recursive Agent Harness studies recursion over full harnesses as a scaling strategy (Lumer et al., 2026), and self-referential agents take this further by letting the loop rewrite itself (Yin et al., 2024); PAD achieves a similar fan-out across *machines* without recursion, because the hub is the shared parent for every node.

Delegation and multi-agent coordination. SearchSwarm studies when a main agent should delegate and how much context to return (Lan et al., 2026); goal-aware collaboration work shows verification is the strongest predictor of success but tends to occur late (Shen et al., 2026). MAST catalogues failure modes of multi-agent LLM systems (Cemri et al., 2025). PAD sidesteps the hardest of these by not letting agents talk to each other: all coordination is mediated by the hub as task records, so there is no inter-agent conversation to drift.

Agent architectures. The practical distinction between a fixed workflow and an agent that decides its own steps (Anthropic, 2024) is exactly the line PAD draws when it dispatches goals rather than commands; our experience matches the guidance that deterministic pipelines are better built as workflows and reserved for their own capabilities. The classical decomposition into planning, memory, and action (Sumers et al., 2023; Wang et al., 2023b), and the line of work on interleaving reasoning with acting (Yao et al., 2023), verbal self-reflection (Shinn et al., 2023), skill libraries (Wang et al., 2023a), and tool learning (Schick et al., 2023), all assume a single agent loop that holds its own state. PAD deliberately removes that assumption and asks what remains when the loop is somewhere else.

Protocols and durable execution. MCP standardizes model-to-tool interfaces and is what our brain-side surface speaks (Model Context Protocol, 2025); its security literature motivates keeping the tool surface small and the authority narrow (Bhatt et al., 2025). Durable-execution systems (Temporal Technologies, 2026) and distributed task frameworks (Moritz et al., 2018) solve analogous problems for deterministic workflows. PAD borrows their vocabulary (claim, lease, heartbeat, idempotency key) but targets a setting where the unit of work is an LLM agent whose completion is not mechanically decidable—hence the evidence gate.

3. Architecture

3.1. Two doors, one state

PAD exposes exactly two surfaces, both keyed by a path secret:

- `/pad/<KEY>/mcp` — served over the Model Context Protocol (Model Context Protocol, 2025). Nine tools, all about tasks: read the situation, submit, inspect, cancel, fetch an artifact, fetch an input file. No shell, no filesystem, no machine identifier.
- `/pad/<KEY>/api/...` — a small REST surface for executors: register, heartbeat, claim, report, close.

Both write to one SQLite database. There is no message bus, no cache, no second source of truth. The persistence layer is deliberately boring: five tables (`nodes`, `tasks`, `reports`, `artifacts`, `task_inputs`) and append-only reporting. Idempotency is a client-supplied key on each report, so a node that loses a response can retry without double-counting.

3.2. Capability routing, not machine addressing

The single most consequential design decision is that a task names a *capability*, never a host. An executor declares which capabilities it can serve; the hub matches work to declarations. In our deployment this means a task asking

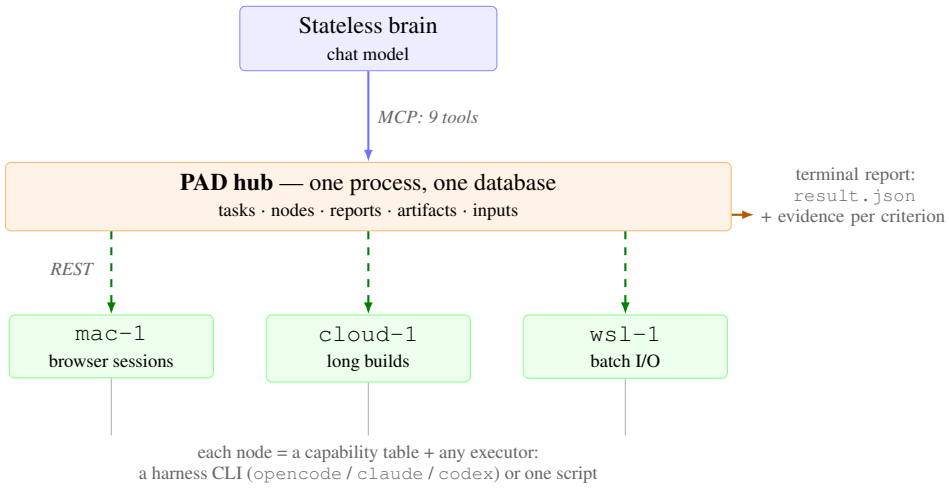


Figure 1. PAD separates the brain, the ledger, and the executors. The brain sees only nine task-management tools and never learns a host name or an operating system. The hub never executes anything and never reasons. An executor is anything that can claim work over REST and write a `result.json`: a plugin-composed harness CLI from one vendor, a different vendor's agent loop, or a deterministic script. Capabilities are matched by name, so the three nodes may be replaced or reordered without the brain changing.

for `scrape.batch` lands on whichever node has offered it, and the brain is never informed which one answered.

Three consequences follow, and all three show up in the data:

1. **Topology is a deployment detail.** Machines were added, renamed, and removed during the measured period with zero changes on the brain side.
2. **Concurrency is an executor decision.** A capability may declare `concurrency: N`, which gives the node N slots; the node only claims work when it has a free slot. Parallelism is therefore a local property of a host, not a scheduling policy the hub has to predict.
3. **Conflict must be declared, not inferred.** The rule we enforce socially and document in the join protocol is: if two machines doing the same thing would collide (shared credentials, shared output file, shared resource), only one may declare that capability. Conversely, read-only work may be declared by many under the same name, which is how the fleet scales horizontally. Renaming a conflicting capability to dodge the rule does not help—the hub would route both.

Note the asymmetry with harness-centric designs: capability routing requires the hub to know *what*, never *where*. That is what makes the brain topology-blind.

3.3. Goals, not steps

A dispatched task carries a goal, a list of constraints, a list of acceptance criteria, an optional deadline, and optional input files. It carries no steps and no command. The executor is told: pursue the goal, stay inside the constraints, and if you must choose, choose a path that ends with evidence.

This is a deliberate bet that the executor—not the dispatcher—knows more about its own environment. Our deployment confirms it in a specific way: the same capability label routes to three different vendors' agent CLIs across six machines, and the dispatch text never mentions which. Adapting a new CLI is one function in the node adapter (§3.5).

3.4. Evidence-gated completion

The reporting contract is where PAD departs most from convention. "Done" is not an exit code. Every terminal report must answer the acceptance criteria one by one:

Field	Meaning
`status`	`done` or `failed`
`summary`	one line, for list views and heartbeats
`report`	prose: conclusion, what was achieved, per-subtask outcome, risks
`acceptance_results`	one entry per criterion: `passed` + `evidence`
`artifacts`	files with name, rows, schema, size, upload flag

The hub computes a coverage record on receipt and attaches it to the report (`missing` lists criteria that were not answered). It judges only *whether a criterion was addressed*, never *whether the answer is true*— truth is left to an independent reviewer node, which is a separate capability. This split matters: an in-loop self-audit (as in (Ma et al., 2026)) and a hub-side coverage check answer different questions

and we use both.

Two secondary rules complete the contract. Data volume is compressed but answers are not: large raw output stays on the node and the hub keeps a pointer with row counts and schema, while fields the dispatcher named explicitly must be reproduced in full. And a failed task must still deliver: partial results, the attempted paths, where it got stuck, and what evidence exists.

3.5. Executor-agnostic nodes

A node is a YAML file and a daemon. Each capability maps to either a shell command or a call into a thin adapter that renders a prompt template and invokes an agent CLI headlessly:

Executor	Invocation
opencode	`opencode run --standalone --auto \--title pad:<id> <prompt>`
claude	`claude -p --dangerously-skip-permissions <prompt>`
codex	`codex exec -C <ws>` plus its bypass-approvals flag, then `<prompt>`
script	any command that writes `result.json`

Because the adapter, not the hub, knows the vendor, machines of different vendors coexist under identical capability semantics. This is the concrete mechanism behind the "heterogeneous" claim, and it is why a fleet can be assembled from hardware the operator already owns.

3.6. Lifecycle properties that had to be engineered

Four behaviors are not obvious from the architecture but are necessary for it to survive contact with reality. Each came from a real incident in the deployment.

Cancellation must reach the execution plane. Marking a task cancelled in the database while its process keeps running holds a slot hostage: on a single-slot node, one cancelled task blocks every queued task behind it. Executors now poll a lightweight status endpoint and kill the whole process group on cancellation.

Timeout is not cancellation. A soft session timeout may fire while a descendant process is still doing valid work. We therefore treat timeout as a lease event—harvest the result if it lands—and reserve process killing for explicit cancellation only.

Receipt is not persistence. A node that receives HTTP 200 has not necessarily landed its report; in one incident every node reported success while the hub stored empty payloads. Nodes now read back their own report and retry until they observe it, using the idempotency key.

Skeleton drift is invisible without a fingerprint. Documents and prompt templates live on the hub and are copied to nodes, so a fixed rule only reaches nodes that re-pull. We hash the behavioral files into a short fingerprint, have nodes report theirs at registration, and surface mismatches in the situation view.

4. Operational Model

4.1. What the brain can do

The nine tools form a closed vocabulary: read state; submit; inspect one task; read an artifact by paged offset; read an input file; cancel. A general chat model can drive this without tool-specific training, and—importantly—without being told anything about the fleet. In practice a session begins with one situation read and proceeds with submissions whose goals reference capabilities the model saw there.

This narrowness is a security property as much as a usability one. The tool surface is the authority surface; a fetch-by-URL path that accepts arbitrary remote content is the only place where the hub touches the network on behalf of an external party, and it is written to refuse private and loopback resolution, re-validate every redirect hop, and abort on size limits.

4.2. Reporting discipline

We require progress reports to be one line and terminal reports to be prose, and we separate the two deliberately. Progress exists so the brain can maintain a world model cheaply; terminal reports exist so a human (or a reviewer node) can audit. Logs and stack traces belong to neither: they stay on the node as artifacts. Compressing volume while keeping pointers is what makes a 42-hour task inspectable without flooding a context window.

4.3. Adding work

A new capability is a YAML entry, not a code change. In the measured deployment this produced a long tail of narrow capabilities (a scheduled content pipeline with ordered dependent steps, a scraping batch job, a per-machine self-check) beside a few broad ones (`*.generic` for judgment work). The tail is a feature: narrow capabilities carry operational knowledge the dispatcher should not have to restate every time.

We note one counter-pattern we hit and document for others: naming a capability after a workflow invites the brain to apply that workflow to goals that merely resemble it. A label like `mac.daily.all` implicitly means "run today's fixed pipeline", so a request to prepare tomorrow's output gets today's date baked in. Our rule is that fixed-workflow capabilities are used only when the intent is literally "as

Table 1. The measured fleet. "Account state" means browser or platform sessions that only one machine may hold; such capabilities are single-owner by the conflict rule of §3.

Node	Caps.	Hours	Role
laptop (macOS)	7	90.2	publishing (browser)
cloud x86	2	239.4	production host, long builds
linux aibook	3	266.9	code & data, two executors
WSL host A	4	145.0	batch I/O, overseas
WSL host B	3	85.9	batch I/O
domestic low-mem	2	7.4	domestic reach, 1.4 GB
total	20	834.8	6 machines, 3 vendor CLIs

Table 2. Duration and outcome by bucket (work tasks only, $n = 938$). Evidence coverage is the fraction of completed tasks whose terminal report answers every acceptance criterion.

Duration	n	done	failed	canc.	evidence
$<$ 10 min	388	295	66	25	89%
10 min–1 h	375	319	31	25	97%
1–6 h	150	112	27	11	96%
$>$ 6 h	25	16	7	2	100%
all	938	742	131	63	93%

usual"; anything novel routes to a generic capability where planning stays with the executor.

5. Deployment Study

5.1. Setting

We instrumented a live deployment over roughly two months. It is not a benchmark: it is the operator's actual work, including the failures. Table 1 describes the fleet at measurement time.

The corpus is 975 tasks, of which we exclude 37 self-check tasks (`*.ping.self`) from the behavioral analysis, leaving **938 work tasks**. Two tasks were still running at measurement. Machine-hours are *occupied* hours, computed as created-to-terminal wall time summed per node; they include queueing, because queueing is what the operator experiences.

5.2. Scale and duration

Table 2 is the headline. Median work duration is 14.5 minutes, but the distribution has a long tail: 18.7% of tasks exceed one hour, 25 exceed six hours, and the maximum is 42.1 hours. Evidence coverage *rises* with duration—89% in the shortest bucket against 97–100% in the rest—which is the opposite of what one would expect if long tasks were where the contract breaks down. Our reading is that long tasks are exactly where executors take the reporting requirement seriously, because a long task that returns without evidence has obviously failed its purpose, whereas a two-minute task can slide by on a one-line "done".

5.3. Where the time goes

Separating queueing from execution, median wait from submission to first report is 13 seconds (p90 112 s, p99 5 987 s) while median execution is 813 s (p90 7 134 s, p99 24 711 s). The p99 queue figure reflects deliberate backlog, not contention: when all compatible nodes are busy, work waits, and the hub reports that honestly instead of failing fast.

Load across nodes is uneven (7–267 h, Table 1) and this is a property of the capability map, not of a load balancer. The low-memory domestic box contributed 0.9% of machine-hours because its declared work is narrow. We consider this correct: the value of a heterogeneous fleet is *reach*, not equal utilization.

5.4. Evidence and the failure path

Of 742 completed work tasks, **693 (93%)** carry per-criterion acceptance evidence. Of 131 failed tasks, **55 (42%)** still carry structured acceptance results and 55 attach artifacts; the average failed report is 862 characters of prose describing what was attempted, what was delivered, and where it broke. This is the property the design was built for: a failure is an artifact, not an absence.

Across the 933 tasks that carried acceptance criteria at all, 734 (79%) produced acceptance results at all, and of those, 483 answered every criterion with no gaps. We report both numbers because they measure different things: whether an executor reported against the criteria, and whether the report left any criterion unanswered. We report the low figure honestly: older tasks predate the mandatory-evidence rule, and the check distinguishes "answered nothing" from "answered wrongly". The temporal trend is what matters—the fraction rose monotonically after the rule became mandatory, and in the current period the completion path virtually always answers every criterion.

5.5. Cancellation propagation

63 tasks were cancelled, 61 of them after emitting at least one progress report. Median interval from the last progress report to the recorded cancellation is 8.8 minutes, which is dominated by long-tail cases where the operator cancelled a task that had gone quiet; the maximum is 35 hours and reflects exactly the failure mode that motivated status polling. We report both figures rather than the median alone: the mechanism works, but a coarse-reporting capability can leave a long silence before anyone notices.

Table 3. Dominant cause of the 132 failed tasks, from a fixed first-match rule over the terminal report text (`tools/classify_failures.py`). Categories are mutually exclusive, so the column sums to 132.

Cause	n
Executor environment (CLI, PATH, crash)	64
Timeout / interruption	16
Goal not achievable as stated	16
Credentials or authorization	15
Unclassified	21

6. Failure Taxonomy

We classified the 132 failed tasks by the dominant cause visible in their terminal reports (Table 3).

The distribution is the strongest argument for the architecture. Nearly half of all failures (64 of 132) are *executor-environment* failures: an agent CLI not on the keep-alive PATH, a crash, a missing binary. A single-loop harness observes these as a broken tool call and can only retry; a hub observes them as a per-node, per-capability pattern and the operator fixes the environment once. This is the difference between debugging an agent and operating a fleet.

Timeouts, unachievable goals, and credential expiry follow at comparable rates. The unachievable-goal class contains the most interesting cases: a licensing dead end, a search backend whose schema had drifted, an authorization judgment that was later judged too broad. Each produced a structured failure report that a human then acted on. This is what makes "failed" a first-class outcome rather than an error.

7. Discussion

What this buys. Three things, empirically. Topology independence: 20 capabilities across 6 machines, 3 vendor CLIs, and the brain's interface never changed. Evidence at scale: 93% of completions audited per criterion, with the fraction rising rather than falling with task length. Operational visibility: because the hub sees every claim and every terminal report, the failure taxonomy of §6 is a query, not an investigation.

What it costs. Three costs, honestly.

First, describing work is the operator's burden. A goal with acceptance criteria takes longer to write than a command, and a vague criterion produces a vague answer. We found no way around this: it is the price of auditability, and it shifts effort from post-hoc debugging to up-front specification.

Second, the hub is a single point of coordination. One SQLite file is excellent for a single operator and a poor fit for multi-tenant or geographically-replicated use. We chose availability-by-simplicity: restarting the hub drops in-flight process supervision, and we schedule that outside working hours rather than pretending it is free.

Third, the separation that makes routing clean makes tight coupling hard. An executor cannot ask the brain a mid-task question, by design. When a task genuinely needs a decision, the executor must either find an authorized path or deliver a failure report and let the next submission carry the decision. We accept the round trip; others may not.

Relation to harness-centric designs. We do not claim PAD replaces an agent harness. In our deployment the per-node executor *is* a harness CLI, and improvements there raise the ceiling of every task PAD routes. The claim is about division of labor: harnesses should make one agent better at long-horizon work; hubs should make many heterogeneous agents composable and auditable. Conflating the two puts durable state inside the process most likely to die.

Threats to validity. Our data comes from a single operator's workflow, so capability names and durations reflect one person's task mix. The evidence-coverage numbers depend on a rule introduced mid-deployment, which we report both overall and by period. Machine-hours are wall-clock occupied time, not compute. We deliberately report the unflattering figures (79% of criteria-carrying tasks reported at all, 52% left no criterion unanswered over all history) alongside the current-period trend rather than quoting only the latter.

8. Conclusion

The interesting problem in long-horizon agents is not making a model plan better; it is making a system remember, route, and prove. PAD is a small, opinionated answer: put all durable state in one hub, let the brain see only nine task tools, route by declared capability so topology never enters the prompt, and refuse to call anything done without evidence per acceptance criterion. A two-month deployment of 938 work tasks across six heterogeneous machines suggests this holds up in the tail—the 19% of tasks over an hour and the handful over a day—where single-loop context management is most fragile. The remaining costs are real and are, in our view, the right ones to pay: specification effort up front, a simple single-point ledger, and no mid-task conversation between executor and brain.

Acknowledgements

We thank the operators of the six nodes for keeping the fleet alive through the incidents catalogued in §4, and the authors of the open harnesses plugged in behind the capability table.

References

Anthropic. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, 2024.

Bhatt, M., Narajala, V. S., and Habler, I. ETDI: Mitigating tool squatting and rug pull attacks in model context protocol by using OAuth-enhanced tool definitions and policy-based access control. *arXiv preprint arXiv:2506.01333*, 2025.

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

Cordis Authors. A programming paradigm for spatiotemporal composability. *arXiv preprint arXiv:2608.25512*, 2026.

DeepSeek-AI. DeepSeek Harness: Everything is a plugin. https://github.com/deepseek-ai/deepseek-harness, 2026. Developer preview; built on Cordis.

Ding, T., Nannapaneni, A., Liu, B., and Zhang, L. Always-on agents: A survey of persistent memory, state, and governance in LLM agents. *arXiv preprint arXiv:2606.30306*, 2026.

Lan, X., Chen, Q., Tao, K., Tang, X., Wang, T., Cao, Q., Kong, X., Wen, Z., Zhang, Z., and Zhou, J. SearchSwarm: Towards delegation intelligence in agentic LLMs for long-horizon deep research. *arXiv preprint arXiv:2606.09730*, 2026.

Lumer, E., Sen, S., Paul, K., and Subbiah, V. K. Recursive agent harnesses. *arXiv preprint arXiv:2606.13643*, 2026.

Ma, Z., Huang, H., Zou, S., Wang, Y., Yang, S., Hu, Y., Wei, F., and Chu, X. LongHorizon-Harness: Advancing long-horizon agents for real-world tasks. *arXiv preprint arXiv:2608.01964*, 2026.

Model Context Protocol. Model context protocol specification, 2025-11-25. https://modelcontextprotocol.io/specification/2025-11-25, 2025.

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

Shen, X., Yan, L., Wang, S., Wu, J., Chen, Z., Tang, J., and Ren, J. Goal-aware dynamic multi-agent collaboration for long-horizon tasks. *arXiv preprint arXiv:2610.39193*, 2026.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.

Temporal Technologies. Temporal: Durable execution platform. https://docs.temporal.io, 2026.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023a.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J.-R. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023b.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Yin, X., Wang, X., Pan, L., Lin, L., Wan, X., and Wang, W. Y. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.

Zheng, J., Fang, X., Zhang, J., Gui, Z., Chen, H., and Zhang, N. OneDayAgent: Towards a long-horizon harness for autonomous agents. *arXiv preprint arXiv:2608.05013*, 2026.

Zhou, C., Chai, H., Chen, W., Guo, Z., Shan, R., Song, Y., Xu, T., Yang, Y., Yu, A., Zhang, W., et al. Externalization in LLM agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*, 2026.

A. The Nine Tools

Tool	Purpose
`get_digest`	fleet state: nodes, capabilities, active tasks, recent per node
`submit_task`	dispatch a goal with constraints, criteria, files
`get_task`	one task: status, report stream, artifacts, input files
`get_artifact`	page through an artifact's bytes by offset
`get_input_file`	open a file dispatched with the task
`close_task`	cancel; propagates to the executor's process group
`probe_file_upload`	verify the attachment path end to end
`get_artifact_delivery`	signed link to an artifact (optional)
`get_artifact_observe`	sampled frames from a media artifact (optional)

B. Reproducing the Measurements

```
python3 tools/collect_stats.py \
        --out paper/data/tasks.json
```

The script reads the situation endpoint with a large recent-task window, fetches each task record concurrently, and prints the aggregates used in Tables 2–3. Task identifiers and goals are operator data; the shipped dataset is aggregated only.